练习题

云计算赛项

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

项目和任务描述

该项目的目标是部署和扩展 Web 应用程序，以支持 Reservations API 和 Web Lookup 服务。

您需要使用预先提供（题目中提供下载链接）的两个基于“go”（Golang）程序的二进制文件部署高度可用、可扩展且高效的 Web 应用程序。这些二进制文件具有不同的服务依赖关系，如下面的 “技术细节” 部分所述。这些应用程序对缓存和水平伸缩可以及时反应。

请将上述文件上传到您自己比赛账户的 S3 存储桶中，以作备份。

⚠️ ***注意：***
这些应用程序无法 “开箱即用”，相反，您将有一段时间来启动初始基础架构。比赛开始后，您的应用将开始在 root 应用程序上接收请求。如果你没有启动有效的解决方案，你将会开始被扣分。

请不要删除环境初始化提供的 Role。

比赛过程中，您需要应对可能出现的任何挑战。挑战不仅仅来自于动态变化的流量，也可能来自于初始架构本身。

初始状态

该 API 目前无法正常工作，但也没有生成任何流量。Requests 将在比赛开始一段时间后，开始发送到您的 API 上。

任务

1. 使用分配的 HashCode 和信息去登陆， Cloud Raiser 平台（当天提供登陆信息）
2. 在登陆时，设置你的名称（格式： 请根据参赛要求）
3. 请仔细阅读文档，对您会大有帮助
4. 登录 AWS 控制台（控制面板提供的链接）
5. 检查环境中的现有配置
6. 按照技术细节中所述配置相关服务器依赖项
7. 您最重要的任务是启动并保证 ROOT 应用可用，可扩展
8. 在 CloudWatch 中配置必要的应用程序监控、指标和警报
9. 在 “Score Events and Scoreboard” 中以及通过 AWS 控制台和 CloudWatch 监控应用程序服务器的性能
10. 通过接收 Requests 得到分数，请参阅“Score Events 和 Scoreboard” 以确保您通过提供的服务获得正面评分
11. 监控成本，不要过度扩展基础架构以最大限度地减少扣分
12. 流量的多少随时间改变并持续发生

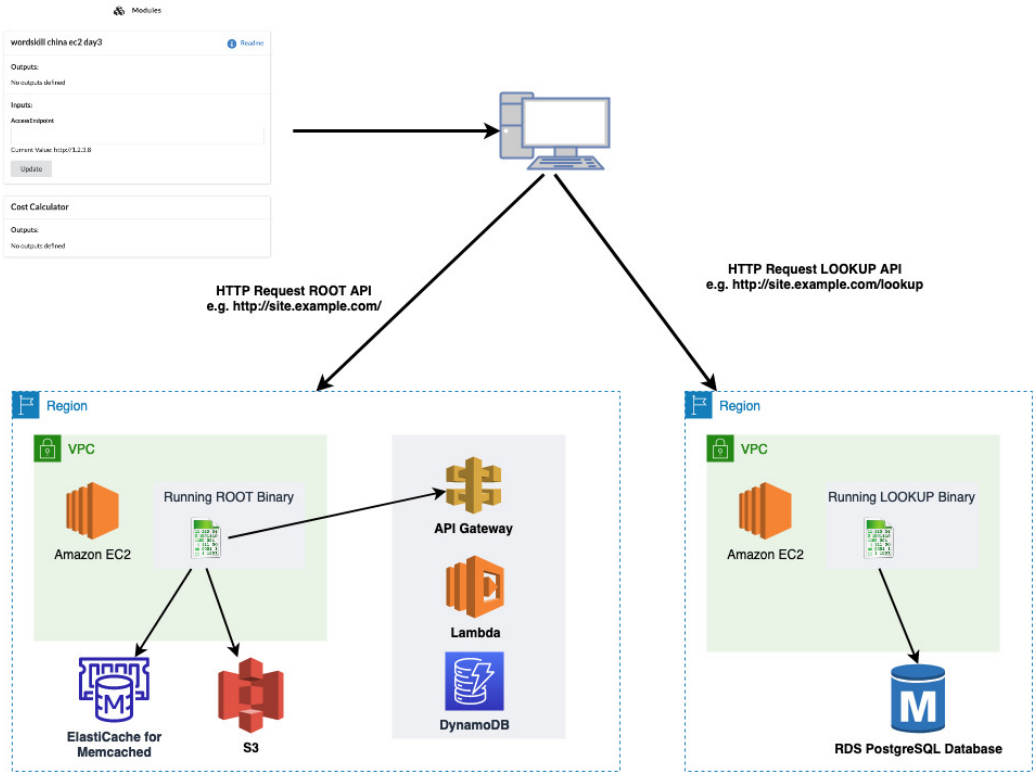
© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

技术细节

1. root server 应用程序部署为从源代码编译的 Go 二进制文件。***请勿以任何方式更改二进制文件，否则将被取消比赛资格。***
2. lookup server 应用程序部署为从源代码编译的 Go 二进制文件。***请勿以任何方式更改二进制文件，否则将被取消比赛资格。***
3. 在今天的竞赛中，使用 *ECS* 作为您的计算资源。
4. root 应用程序在开始变慢之前可以处理大约 5 个连接请求。小心超载，并在队列填满时注意来自服务器的 HTTP 5XX 响应。
5. 服务器应用程序和相关文件将在比赛当天提供给选手。
6. 此版本中的 ROOT 服务器有许多其他要求才能运行。除了上面列出的基本要求外，还要满足如下其他服务要求。
    a. 它必须具有侦听定义的 TCP 端口（默认端口 80）的权限。
    b. 它必须具有以 “server.ini” 形式提供的配置文件。
    c. 它必须能够访问正在运行的 Memcached 服务器。
    d. 它必须能够访问 API-Gateway。
    e. 服务器必须能够写入日志文件。该文件名为 “app_logs”，位于服务器配置文件中提供的目录中。
    f. 它必须有向 S3 存储桶上传文件的权限。
7. 此版本中的 LOOKUP 服务器有许多其他要求才能运行。除了上面列出的基本要求外，还要满足如下其他服务要求。
    a. 它必须具有侦听定义的 TCP 端口（默认端口 80）的权限
    b. 它必须具有以 “server.ini” 形式提供的配置文件
    c. 它必须能够访问正在运行的 PostgreSQL 服务器。
    d. 服务器必须能够写入日志文件。该文件名为 “app_logs”，位于服务器配置文件中提供的目录中。
8. 这些服务器需要连接到 Memcached 数据库和 PostgreSQL 数据库才能存储数据。运行基础架构的效率越高，服务器响应请求的速度就越快，获得的积分也就越多。
9. 选择的基本操作系统是 ***Amazon Linux Version 2*** (https://aws.amazon.com/amazon-linux-ami/). 该发行版之所以被选中，是因为它具有广泛的行业支持、稳定性、可用性以及与 AWS 的出色集成。
10. 使用 AWS CLI（命令行界面）可能非常有用。有关在本地安装此工具的信息，请访问 http://docs.aws.amazon.com/cli/latest/userguide/installing.html. AWS CLI 已安装在 Amazon Linux EC2 实例上。

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

参考架构



上面的示例说明了部署应用程序的一种可能的架构设计，**仅供参考**。

服务细节

概述

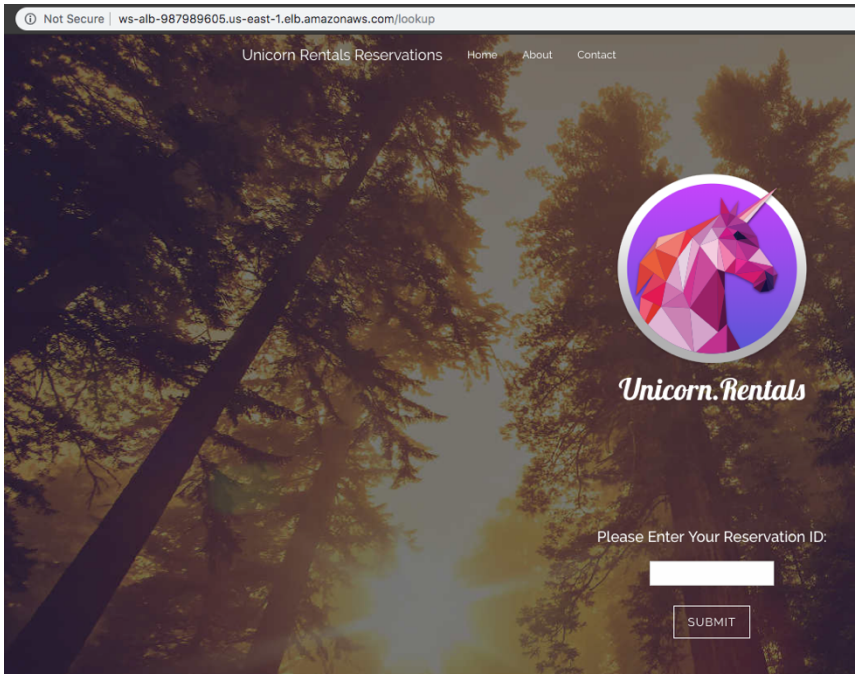
一旦您搭建的服务器收到了请求，响应所需的数据会存储在数据库中。如果您收到了请求并且之前已经响应过该请求，则不会创建新的数据。如果您的服务器对每种技术都使用中心化的解决方案，那么无论哪个实例响应了请求，所有实例都可以访问响应数据。但是，如果您在每个实例上都部署了数据库服务器，则响应答案将不会被共享，每个服务器最终都必须为每个请求创建响应。

URL 路由

您将需要提供单个 URL 网址作为答案，该网址将被用来访问您的两个应用程序。为了便于执行此操作，您需要执行 URL 路由。标识为 “root” 的应用程序应在您的 URL 的根目录下运行。因此，如果部署得当，当您访问应用程序的公共 URL 时，您应该会看到以下屏幕：

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

您需要将第二个名为 “lookup” 的应用程序部署到主网址之外的 “/lookup” URL。如果你的主网址是 http://server.example.com, 那么在浏览器中点击该主网站应该会返回上面的图片。但是，前往 http://server.example.com/lookup 应该显示以下页面：



你也可以通过将名为 “lookup” 的应用程序部署到你的 URL 的 “/lookup” 后缀来实现这一点。您输入的 URL 将在 URL 的根目录以及 “/lookup” 终端节点上接收请求。部署应用程序后，将在数据库服务器上加载默认预定信息。示例可以在下面找到。如果您的安装正确，您应该能够查询到这些预定信息，输入下面列出的预定 ID 值之一：

```
5000001
5000002
5000003
```

Root 应用程序 — Memcached 数据库

您需要为根应用程序提供 Memcached 数据库解决方案。这不是 I/O 密集型工作负载，应尽可能以最经济高效的方式进行部署。

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

完成后，您需要通过部署到每个实例的 “server.ini” 文件向每台服务器说明如何使用此服务。与 Memcached 相关的配置部分如下所示

```
"MemcacheHost" = "127.0.0.1"
"MemcachePort" = "11211"
```

MemcacheHost 是用于连接到 MemcacheHost 服务的主机名。
MemcachePort 是用于连接到服务的 TCP 端口号（默认值为 11211）。

Root 应用程序 — S3 存储桶
您的应用程序将需要访问 S3 并上传预定信息（退款请求）的权限，一个名称格式为 ‘xxx-EC2InstanceProfile-xxx’ 的 EC2 Profile 将存在于您的 AWS 账户中供您使用。

您需要通过部署到每个实例的 “server.ini” 文件向每台服务器说明预定信息（退款请求）将被上传到那个 S3 存储桶中。

```
"AwsS3Bucket" = "aaa-bbb-ccc-ddd-eee-xxxyyyzzz"
```

Root 应用程序 — API Gateway + Lambda + DynamoDB
您的应用程序将需要访问 API GATEWAY 端点，该端点会出发一个已经在您的账户中创建好的 Lambda 函数（名称格式为 ‘xxx-LambdaFetchDynamoDB-xxx’），该 Lambda 函数将会访问一个需要您创建的 DynamoDB 表（表名：**ws-china-table**），该表的分区键（Partition_key）为 **account_id**，值为您的 AWS 账户 ID。

您需要通过部署到每个实例的 “server.ini” 文件向每台服务器说明哪个 API Gateway 端点将会被访问。

```
"APIGWUrl" = "https://abcl23.execute-api.us-east-1.amazonaws.com/v1/"
```

Lookup 应用程序结构
在一天中，您部署的预订查询系统将从外部系统接收新的预订数据。您必须确保您的申请可用，因为这些预订只会收到一次。如果您的申请未收到此信息，则不会再次发送。Unicorn Rentals 的客户会定期使用您部署的系统查找他们的预订详情。如果他们的预订未在系统中列出，您将被扣除积分。

Lookup 应用程序 - PostgreSQL
您需要为应用程序提供 PostgreSQL 数据库解决方案。这不是 I/O 密集型工作负载，应尽可能以最经济高效的方式进行部署。部署了 PostgreSQL 服务后，您需要创建为请求提供服务所必需的表。必须创建两个表，一个用于网站内容管理系统，另一个用于存储独角兽租赁预订信息的表。可以通过 database.sql 文件找到表定义示例，以及这些表的默认数据。

完成后，您需要通过部署到每个实例的 “server.ini” 文件向每台服务器说明如何使用

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.

此服务。与 PostgreSQL 相关的配置部分如下所示：

```
"PgsqlHost" = "localhost"
"PgsqlPort" = "5432"
"PgsqlUser" = "unicorndb"
"PgsqlPass" = "unicorndb"
"PgsqlDb" = "unicorndb"
```

PgsqlHost 是用于连接到 PostgreSQL 服务的主机名。
pgsqlPort 是用于连接到服务的 TCP 端口号（默认值为 5432）。
pgsqlUser 是用于连接到服务的用户名。使用根用户名不是最佳做法。
PGSQLPass 是分配给用户（如上所述）用于访问此服务的密码。
pgsqldb 是您创建的新表所在的数据库的名称。您很可能还需要创建此数据库名称。

Lookup 应用程序 – 安全
最近发现我们的预订查找系统容易受到 SQL 注入攻击，并发现有人正在使用此漏洞破坏我们的数据。 他们一直在将我们客户租赁的独角兽访问代码更改为“H@XXORR333DDD”。 创建应用程序的开发人员不再可用，因此我们希望您能够找到一种方法来确保应用程序的可用性。

将会发送到您应用上（假如您的 URL 是 http://server.example.com）的请求：
将会发送到 ROOT 应用上的请求:

1. http://server.example.com
2. http://server.example.com/healthcheck
3. http://server.example.com/calc

将会发送到 LOOKUP 应用上的请求:

1. http://server.example.com/lookup
2. http://server.example.com/lookup/add
3. http://server.example.com/add
4. http://server.example.com/search
5. http://server.example.com/lookup/search

© 2023, Amazon Web Services, Inc. or its affiliates.
All rights reserved.